

Laser X-Z Controller

UART Interface and Motion Control Specification

Firmware: LASER_XZ 2.4 **Controller:** STM32F401RCT6 **Motor interface:** Two STEP/DIR drivers with shared enable **Document purpose:** Independent GUI and host-software implementation **Document date:** 2026-08-05

This document is the complete host-side interface contract for developers who need to create another GUI, test application, script, or embedded host for the Laser X-Z positioning controller. It describes the electrical UART connection, byte encoding, commands, replies, controller state, motion conversion, acceleration behavior, resolution, limits, error handling, and recommended GUI workflow.

The firmware source remains the final authority if a later firmware version changes behavior. A host should verify the version using [PING](#) before enabling motion.

1. Interface summary

Property	Value
UART peripheral	STM32 USART1
STM32 receive pin	PA10
STM32 transmit pin	PA9
Logic level	3.3 V UART TTL
Baud rate	115200 bit/s
Framing	8 data bits, no parity, 1 stop bit (8N1)
Flow control	None
Protocol type	Newline-delimited ASCII text
Host command terminator	LF (0x0A); CR+LF is also accepted
Controller reply terminator	CR+LF (0x0D 0x0A)
Maximum command length	159 characters before LF
Axes	X and Z
Position units	mm for normal commands; raw pulses for legacy commands
Default scale	X = 10 pulses/mm, Z = 10 pulses/mm
Physical increment	0.1 mm per STEP pulse on each axis
Conservative real positioning increment	0.2 mm until measured repeatability proves better
Default maximum speed	20 mm/s per axis
Default acceleration	40 mm/s ² per axis
Enforced SPEED_MM range	1 to 100 mm/s
Enforced ACCEL_MM range	5 to 200 mm/s ²
Position feedback	Open-loop commanded STEP count, not an encoder
Homing inputs	None
Software travel limits	None
Checksum or CRC	None
Command sequence number	None

1.1 Electrical wiring

Use a USB-UART adapter or host UART with 3.3 V logic.

Host or USB-UART	STM32F401RCT6
Host TX	PA10 / USART1 RX
Host RX	PA9 / USART1 TX
Host GND	Controller and driver common GND

TX and RX must be crossed. Do not connect an RS-232 port directly because its voltage levels are incompatible. Normally the UART adapter VCC is not connected; power the controller through its intended board power input.

2. Data encoding on the serial wire

The controller does not use binary packets. Every command and reply is readable ASCII text. Numbers are sent as base-10 characters. For example, **-2.5** is sent as four bytes representing **-**, **2**, **.**, and **5**; it is not sent as a binary integer or IEEE-754 float.

The GUI sends this command:

```
MOVE_MM X 1 Z -2.5<LF>
```

Its exact UART payload is:

Representation	Bytes
ASCII	M O V E _ M M [space] X [space] 1 [space] Z [space] - 2 . 5 [LF]
Hexadecimal	4D 4F 56 45 5F 4D 4D 20 58 20 31 20 5A 20 2D 32 2E 35 0A
Decimal bytes	77 79 86 69 95 77 77 32 88 32 49 32 90 32 45 50 46 53 10

The written number **1** is ASCII byte **0x31** (49 decimal), not byte **0x01**. The firmware uses Arduino `println()` for replies, so controller lines end in CR+LF. The acknowledgement **OK MOVE_MM<CR><LF>** is:

```
4F 4B 20 4D 4F 56 45 5F 4D 4D 0D 0A
```

At the electrical level, 8N1 adds a low start bit, sends each byte least significant bit first, then adds one high stop bit. These framing bits are handled by the UART hardware and are not part of the application byte array.

2.1 Line rules

- End every command with LF (`\n`, hex `0A`).
- The firmware ignores CR (`\r`, hex `0D`), so CR+LF is safe.
- Controller replies use CR+LF; host line readers should tolerate LF or CR+LF.
- Commands are case-insensitive because the firmware converts them to uppercase.
- Spaces, tabs, and commas are accepted as token separators.
- Use a period for decimal values, for example `12.5`.
- Send one command per line.
- Do not include a checksum, length prefix, JSON, or binary header.
- Parse every received line independently because events are asynchronous.
- A line that exceeds 159 characters is discarded and produces an error.

3. Startup and connection handshake

After reset, the controller disables both drivers and transmits two unsolicited lines:

```
READY LASER_XZ 2.4
STATUS ENABLED=0 ESTOP=0 FAULT=NONE BUSY=0 X=0 XT=0 XH=0 XMM=0.0000 XTMM=0.0000 XSPMM=10.0000 XVMM=20.000 XAMM=40.000
Z=0 ZT=0 ZH=0 ZMM=0.0000 ZTMM=0.0000 ZSPMM=10.0000 ZVMM=20.000 ZAMM=40.000
```

A host must not depend on receiving **READY**; it may open the port after the message was already sent. Use this handshake instead:

```
Host -> PING
STM32 -> OK PONG LASER_XZ 2.4
Host -> STATUS
STM32 -> STATUS ...
```

Recommended validation:

1. Open the port at 115200 8N1 with no flow control.
2. Clear stale bytes from the host input buffer if appropriate.
3. Send **PING\n**.
4. Require **OK PONG LASER_XZ 2.4** within approximately 2.5 seconds.
5. Accept the firmware only if the major version is compatible.

6. Send **STATUS** and build the GUI state from the returned fields.
7. Keep movement controls disabled until **ENABLED=1**, **ESTOP=0**, and **FAULT=NONE**.

4. General reply model

The controller emits complete newline-terminated lines in four categories.

Prefix	Meaning
READY	Unsolicited startup identity.
OK	A command was accepted or completed immediately.
ERR	A command was rejected; no requested motion was started.
EVENT	An asynchronous state transition.
STATUS	A complete state snapshot requested by the host or sent at startup.

There is no transaction ID. A host should normally send one configuration or motion command, consume its **OK** or **ERR** reply, and then continue. It must still handle an **EVENT** line at any time. The first **ESTOP** command is a special case: the firmware sends the event before its acknowledgement.

5. Complete command reference

5.1 Identification and state commands

Host command	Successful controller reply	Description
PING	OK PONG LASER_XZ 2.4	Identify compatible firmware.
STATUS	STATUS key=value ...	Return a full state snapshot.
HELP	OK COMMANDS=PING, STATUS, ENABLE, DISABLE, STOP, ESTOP, CLEAR, MOVE_MM, GOTO_MM, SPEED_MM, ACCEL_MM, ZERO, MOVE, GOTO, SPEED, ACCEL	Return the compact command-name list.

5.2 Driver and safety commands

Host command	Successful controller reply	Effect
ENABLE	OK ENABLE	Drive shared PB10 active-low and enable both motor drivers. Rejected while faulted.
STOP	OK STOP	Immediately set both targets to their present pulse counts. Drivers remain enabled and coordinate flags remain unchanged.
DISABLE	OK DISABLE	Stop immediately, remove holding torque, and clear XH and ZH reference flags.
ESTOP	EVENT ESTOP SOURCE=SERIAL, then OK ESTOP	Latch software E-stop, stop, and disable both drivers.
CLEAR	OK CLEAR	Stop and clear the software E-stop/fault latch. Drivers remain disabled after an E-stop.
ZERO X	OK ZERO	Set current and target X pulse positions to zero without moving. Idle only.
ZERO Z	OK ZERO	Set current and target Z pulse positions to zero without moving. Idle only.
ZERO ALL	OK ZERO	Set both current and target pulse positions to zero without moving. Idle only.
HOME X	ERR UNSUPPORTED this controller has no homing inputs	Always unsupported for valid X, Z, or ALL syntax.

Important behavior:

- **STOP** is an immediate pulse stop, not a controlled deceleration.

- If **STOP**, **DISABLE**, or **CLEAR** interrupts active motion, its **OK** reply is normally followed by **EVENT IDLE X=<pulses> Z=<pulses>**.
- **DISABLE** can allow a vertical axis to fall because holding torque is removed.
- **ESTOP** is serial software control, not a safety-rated removal of motor power.
- If **ESTOP** interrupts motion, the usual order is **EVENT ESTOP**, **OK ESTOP**, then **EVENT IDLE**. An already-latched **ESTOP** produces only **OK ESTOP**.
- Firmware 2.4 does not clear XH/ZH inside **ESTOP**; nevertheless, a host should treat the reference as untrusted after holding torque is removed and require the operator to set zero again.
- **CLEAR** does not automatically enable the drivers. Send **ENABLE** separately.

5.3 Relative movement in millimetres

Host command	Immediate reply	Later reply
<code>MOVE_MM X <distance></code>	<code>OK MOVE_MM</code>	<code>EVENT IDLE X=<pulses> Z=<pulses></code> when all active axes finish.
<code>MOVE_MM Z <distance></code>	<code>OK MOVE_MM</code>	Same idle event.
<code>MOVE_MM X <distance> Z <distance></code>	<code>OK MOVE_MM</code>	One idle event after both axes finish.

Examples:

```
MOVE_MM X 1
MOVE_MM Z -0.5
MOVE_MM X 25 Z 10
```

MOVE_MM calculates each target from the current pulse position at the instant the command is parsed:

```
require distance_mm * pulses_per_mm to be a whole pulse
target_pulses = current_pulses + (distance_mm * pulses_per_mm)
```

If a new move is sent while an axis is already moving, it is not queued. It replaces the previous target, and a relative value is based on the instantaneous current position rather than the old target. A GUI should normally wait until idle unless deliberate mid-motion retargeting is required.

5.4 Absolute movement in millimetres

Host command	Immediate reply	Later reply
<code>GOTO_MM X <coordinate></code>	<code>OK GOTO_MM</code>	<code>EVENT IDLE X=<pulses> Z=<pulses></code> when motion finishes.
<code>GOTO_MM Z <coordinate></code>	<code>OK GOTO_MM</code>	Same idle event.
<code>GOTO_MM X <coordinate> Z <coordinate></code>	<code>OK GOTO_MM</code>	One idle event after both axes finish.

Examples:

```
GOTO_MM X 120
GOTO_MM Z 50
GOTO_MM X 120 Z 50
```

Absolute targets are referenced to the software coordinate established by **ZERO**. The firmware reports XH and ZH but does not reject absolute moves when a reference flag is zero. A safe GUI should disable absolute moves until the relevant axis has been zeroed.

```
require coordinate_mm * pulses_per_mm to be a whole pulse
target_pulses = coordinate_mm * pulses_per_mm
```

5.5 Speed and acceleration configuration

Host command	Successful reply	Meaning
SPEED_MM X <mm/s>	OK SPEED_MM	Set X maximum speed.
SPEED_MM Z <mm/s>	OK SPEED_MM	Set Z maximum speed.
SPEED_MM ALL <mm/s>	OK SPEED_MM	Set both maximum speeds.
ACCEL_MM X <mm/s^2>	OK ACCEL_MM	Set X acceleration magnitude.
ACCEL_MM Z <mm/s^2>	OK ACCEL_MM	Set Z acceleration magnitude.
ACCEL_MM ALL <mm/s^2>	OK ACCEL_MM	Set both acceleration magnitudes.

Examples:

```
SPEED_MM ALL 20
ACCEL_MM ALL 40
SPEED_MM Z 10
ACCEL_MM Z 25
```

The values are stored in RAM and remain active until changed or the controller resets. They may be configured while drivers are disabled. Firmware also allows them to change during motion, but a GUI should normally change them only while idle for predictable trajectories.

5.6 Legacy raw-pulse commands

These commands are supported for commissioning and backward compatibility. New GUIs should use the millimetre commands.

Command	Unit	Reply	
`MOVE X	Z <signed_integer>`	Relative STEP pulses	OK MOVE
`GOTO X	Z <integer> [other_axis <integer>]`	Absolute STEP pulse coordinates	OK GOTO
`SPEED X	Z	ALL <value>`	Pulses/s
`ACCEL X	Z	ALL <value>`	Pulses/s^2

At the current scale, 10 raw pulses correspond to 1.0 mm.

6. STATUS reply specification

Example:

```
STATUS ENABLED=1 ESTOP=0 FAULT=NONE BUSY=0 X=100 XT=100 XH=1 XMM=10.0000 XTMM=10.0000 XSPMM=10.0000 XVMM=20.000
XAMM=40.000 Z=50 ZT=50 ZH=1 ZMM=5.0000 ZTMM=5.0000 ZSPMM=10.0000 ZVMM=20.000 ZAMM=40.000
```

Parse the line as space-separated **KEY=VALUE** fields. A forward-compatible GUI should ignore unknown fields and tolerate a different field order.

Field	Type	Meaning
ENABLED	0 or 1	Shared driver enable output state.
ESTOP	0 or 1	Software E-stop latch.
FAULT	text	NONE or active fault code; currently ESTOP .
BUSY	0 or 1	One or both axes have a nonzero step interval.
X, Z	signed integer	Current commanded pulse positions.
XT, ZT	signed integer	Current target pulse positions.
XH, ZH	0 or 1	Software reference flags set by ZERO .
XMM, ZMM	decimal	Current positions in mm.

Field	Type	Meaning
XTMM, ZTMM	decimal	Target positions in mm.
XSPMM, ZSPMM	decimal	Compiled STEP pulses per mm.
XVMM, ZVMM	decimal	Configured maximum speeds in mm/s.
XAMM, ZAMM	decimal	Configured acceleration magnitudes in mm/s ² .

X and **Z** are open-loop command counters. They confirm generated pulses, not actual mechanical position. A stalled motor, loose pulley, slipping belt, or unpowered movement is not detected.

7. Motion conversion and smallest movement

7.1 Mechanical calibration

Both axes are configured for a 1.8 degree motor in half-step mode with a 20-tooth GT2 pulley and 2 mm belt pitch:

```
200 full steps/revolution * 2 half-steps/full-step = 400 pulses/revolution
20 teeth * 2 mm/tooth = 40 mm/revolution
400 pulses/revolution / 40 mm/revolution = 10 pulses/mm
```

Therefore:

```
1 pulse = 1 / 10 mm = 0.1 mm
```

The smallest physical commanded increment is **0.1 mm** on X and Z. A smaller command cannot create a fraction of a STEP pulse. This is command resolution, not guaranteed positioning accuracy. Belt compliance, pulley eccentricity, V-wheel clearance, load, motor half-step nonlinearity, and missed steps affect the real mechanism. Until repeatability is measured on the assembled stage, use **0.2 mm as the conservative real positioning increment** and 0.1 mm only for fine commissioning jogs.

7.2 Enforced movement grid

Firmware 2.4 requires every MOVE_MM distance and GOTO_MM coordinate to align with a whole STEP pulse. At the current calibration, valid values are integer multiples of 0.1 mm. The controller rejects off-grid values instead of silently rounding them to a different physical movement.

Requested relative distance	Converted pulses	Commanded physical movement
0.00 mm	0	0.0 mm
0.01 mm	0	Rejected with ERR RESOLUTION
0.05 mm	Not on grid	Rejected with ERR RESOLUTION
0.10 mm	1	0.1 mm
0.15 mm	Not on grid	Rejected with ERR RESOLUTION
0.20 mm	2	0.2 mm
-0.10 mm	-1	-0.1 mm

A third-party GUI should validate the 0.1 mm grid locally for immediate user feedback. It must still handle the controller's resolution error because the firmware is the final authority.

7.3 Calibration correction

If a commanded 10.0 mm move measures a different distance:

```
new_pulses_per_mm = old_pulses_per_mm * commanded_mm / measured_mm
```

Calibration is compiled into firmware; it cannot be changed through the current UART protocol. Read [XSPMM](#) and [ZSPMM](#) from STATUS rather than assuming 10 forever.

8. Speed behavior

SPEED_MM sets the maximum STEP pulse frequency converted into linear units:

```
pulse_rate_pulses_per_s = speed_mm_per_s * pulses_per_mm
speed_mm_per_s = pulse_rate_pulses_per_s / pulses_per_mm
```

At 10 pulses/mm:

Linear speed	Pulse rate	Pulse interval at constant speed
0.1 mm/s	1 pulse/s	1.000 s
1 mm/s	10 pulses/s	100 ms
5 mm/s	50 pulses/s	20 ms
20 mm/s (default)	200 pulses/s	5 ms
100 mm/s	1000 pulses/s	1 ms

The configured value is a maximum, not an instant speed. The axis ramps toward it according to acceleration and begins decelerating early enough to stop at the target. Short moves never reach the configured maximum.

XVMM and **ZVMM** in STATUS report these configured maximum values. Firmware 2.4 does not report instantaneous velocity, so a GUI must not label XVMM/ZVMM as the present motor speed.

8.1 Enforced real operating speed range

Firmware 2.4 enforces the following range on **SPEED_MM**:

Parameter	Enforced value
Minimum accepted speed	1 mm/s
Default speed	20 mm/s
Maximum accepted speed	100 mm/s

At 10 pulses/mm this is 10 to 1000 half-step pulses/s. The maximum corresponds to 500 full steps/s, 150 motor RPM, and 100 mm/s through the 40 mm/revolution GT2 pulley. The supplied NEMA 17 continuous-duty curves show that a 12 V motor still has useful torque near 500 full steps/s but loses torque increasingly at higher speeds. The 100 mm/s limit therefore preserves margin for the vertical load, belt friction, V-wheels, and cable carrier.

The datasheet curves were measured at 2 A RMS. If the actual driver current is set below 2 A, available torque is lower. Therefore 100 mm/s is a conservative commissioning ceiling, not a guarantee for every payload. Begin at 20 mm/s and increase only after repeated full-travel tests without stalls or missed steps.

9. Acceleration behavior

ACCEL_MM sets the rate at which velocity magnitude changes:

```
pulse_acceleration = acceleration_mm_per_s2 * pulses_per_mm
```

At the default 40 mm/s² and 10 pulses/mm, the controller uses 400 pulses/s². Starting from rest, an ideal continuous model reaches 20 mm/s in:

```
time = speed / acceleration = 20 / 40 = 0.5 s
```

The distance required to accelerate from rest to 20 mm/s is:

```
distance = speed^2 / (2 * acceleration)
distance = 20^2 / (2 * 40) = 5 mm
```

The controller needs another 5 mm to decelerate, so a move needs approximately 10 mm to reach the default maximum speed. Shorter moves use a triangular speed profile. Longer moves use a trapezoidal profile with a constant-speed section.

9.1 Motion-time estimates

For a rest-to-rest move of distance d , maximum speed v , and acceleration a , the following ideal equations are useful for GUI estimates.

If $d < v^2/a$, the move is triangular:

```
peak_speed = sqrt(d * a)
time = 2 * sqrt(d / a)
```

If $d \geq v^2/a$, the move is trapezoidal:

```
acceleration_distance_each_end = v^2 / (2*a)
time = d/v + v/a
```

Default examples:

Move	Profile	Ideal estimate
1 mm	Triangular; peak about 6.32 mm/s	about 0.316 s
10 mm	Just reaches 20 mm/s	about 1.0 s
100 mm	5 mm accelerate, 90 mm cruise, 5 mm decelerate	about 5.5 s

These are estimates. Discrete pulses, scheduler timing, UART activity, and motor mechanics introduce small differences.

9.2 Ramp implementation

The firmware uses a real-time discrete step-interval ramp. It continuously calculates stopping distance as approximately $\text{speed}^2 / (2 * \text{acceleration})$. When the remaining pulses are no greater than the required stopping pulses, it changes from acceleration to deceleration. STEP is high for 4 microseconds and DIR is allowed 1 microsecond of setup time before STEP rises.

Internally, speed and acceleration are first converted to pulses/s and pulses/s². For acceleration a in pulses/s², the initial interval is:

```
c0_us = 0.676 * sqrt(2/a) * 1,000,000
```

Successive step intervals use the discrete recurrence:

```
c_next = c_current - (2*c_current)/(4*n + 1)
minimum_interval_us = 1,000,000 / configured_max_pulse_rate
```

The interval is never allowed below the configured maximum-speed interval or below 6 microseconds. The signed ramp index changes the same recurrence into a deceleration sequence. This is why speed changes smoothly pulse by pulse rather than jumping immediately to `SPEED_MM`.

9.3 Enforced real operating acceleration range

Firmware 2.4 enforces the following range on `ACCEL_MM`:

Parameter	Enforced value
Minimum accepted acceleration	5 mm/s ²
Default acceleration	40 mm/s ²
Maximum accepted acceleration	200 mm/s ²

At 10 pulses/mm this is 50 to 2000 pulses/s². At the maximum combination of 100 mm/s and 200 mm/s², acceleration takes 0.5 s and 25 mm; another 25 mm is needed to decelerate. This fits the 200 mm working envelope while avoiding the unrealistic electrical parser ceiling previously exposed by the interface.

Begin at the 40 mm/s² default. A heavy Z carriage, reduced motor-current setting, tight wheels, belt tension, or cable drag may require a lower value.

10. Two-axis movement

To start X and Z from one parsed command, put both axes in the same line:

```
MOVE_MM X 50 Z 25
GOTO_MM X 100 Z 80
```

Both targets are committed by that command, but each axis has an independent acceleration ramp and maximum speed. The firmware does not perform CNC linear interpolation and does not guarantee that both axes arrive simultaneously. The path in X-Z space may not be a straight line.

BUSY=1 while at least one axis is running. **EVENT IDLE** is generated only after both axes are idle. Its X and Z values are raw pulse coordinates.

If coordinated arrival or a straight-line path is required, it must be added to firmware. Sending many tiny segments from a GUI is not recommended because UART latency and the absence of a motion queue make the path unreliable.

11. Position, limits, and reference behavior

- Position is a signed 32-bit pulse counter on this STM32 target.
- Negative coordinates are accepted.
- There are no firmware soft limits for 0 to 200 mm.
- The GUI's 200 x 200 mm drawing is a display envelope only.
- There are no limit-switch or home-switch inputs.
- The controller cannot detect a crash, stall, skipped step, belt slip, or movement while disabled.
- **ZERO** defines the software coordinate without physical movement.
- **STOP** preserves the pulse coordinate and reference flag.
- **DISABLE** clears reference flags because holding torque is removed.
- After any possible physical movement while unpowered or disabled, the operator must reposition the mechanism and issue **ZERO** again.

A third-party GUI should implement configurable soft limits only after the operator has established a trustworthy zero. Software limits do not replace physical hard stops.

12. Events

12.1 Motion complete

```
EVENT IDLE X=<current_x_pulses> Z=<current_z_pulses>
```

This is emitted when the global busy state changes from moving to idle. After receiving it, request **STATUS** to obtain mm values, target values, dynamics, and all safety fields.

No idle event is guaranteed for a command that creates no movement, such as a relative zero-pulse target. The immediate **OK** still confirms acceptance.

12.2 Software E-stop

```
EVENT ESTOP SOURCE=SERIAL
```

This is emitted when the E-stop latch first becomes active. The first **ESTOP** command therefore normally produces:

```
EVENT ESTOP SOURCE=SERIAL
OK ESTOP
```

13. Complete error reference

Error reply	Cause	Recommended host action
ERR DISABLED send ENABLE first	Movement requested while drivers are disabled.	Send STATUS, then ENABLE if safe.
ERR FAULTED ESTOP	Movement or ENABLE requested while E-stop/fault is latched.	Send CLEAR, then ENABLE, then STATUS.
ERR SYNTAX ...	Wrong number/order of tokens, too many tokens, or ZERO attempted while busy.	Show the detail and correct the command.
ERR AXIS <value>	Axis was not X, Z, or allowed ALL; may also report duplicate axis.	Correct the axis field.
ERR NUMBER <value>	Integer or decimal value could not be parsed completely.	Use finite base-10 text with a period.
ERR RANGE millimetre value is outside position range	mm value cannot convert to signed pulse position.	Reduce the magnitude.
ERR RANGE relative target is outside position range	Current plus relative pulses overflows the signed counter.	Use a valid target/reset coordinate.
ERR RANGE value must be positive	Speed or acceleration is zero, negative, or invalid.	Send a positive finite value.
ERR RANGE speed must be 1..100 mm/s	SPEED_MM is outside the enforced operating envelope.	Use 1 through 100 mm/s.
ERR RANGE acceleration must be 5..200 mm/s^2	ACCEL_MM is outside the enforced operating envelope.	Use 5 through 200 mm/s^2.
ERR RANGE value exceeds the calibrated pulse-rate limit	Speed exceeds 12000 pulses/s or acceleration exceeds 100000 pulses/s^2.	Reduce the physical value.
ERR RANGE speed must be 1..12000	Legacy raw SPEED value is outside its pulse/s range.	Use 1 through 12000 pulses/s.
ERR RANGE acceleration must be 1..100000	Legacy raw ACCEL value is outside its pulse/s^2 range.	Use 1 through 100000 pulses/s^2.
ERR RESOLUTION distance is smaller than one half-step	Nonzero mm value rounded to zero pulses.	Use at least a 0.1 mm GUI increment.
ERR RESOLUTION value must use whole STEP-pulse increments	MOVE_MM or GOTO_MM value is off the compiled movement grid.	Use multiples of 0.1 mm with the current calibration.
ERR RESOLUTION value is below one pulse per second	Speed or acceleration converts below 1 pulse/s or pulse/s^2.	Use at least 0.1 in current physical units.
ERR UNSUPPORTED this controller has no homing inputs	HOME requested with valid syntax.	Use manual positioning plus ZERO.
ERR UNKNOWN_COMMAND <name>	Command name is not implemented.	Check firmware version and spelling.
ERR LINE_TOO_LONG maximum 159 characters	Input line exceeded the receive buffer.	Discard the transaction and send a shorter single command.

Treat every **ERR** as a rejected transaction. Do not assume movement occurred. Request STATUS after errors that concern state, enable, E-stop, or motion.

14. Recommended GUI state machine

14.1 Disconnected

- All controller and motion buttons disabled except Connect/Refresh.
- Open the selected port at 115200 8N1.

14.2 Connected but unverified

- Send PING and STATUS.
- Accept only compatible OK PONG LASER_XZ 2.x firmware.
- Do not unlock motion based only on the COM port opening successfully.

14.3 Verified but motion locked

Enable jog and absolute controls only when all conditions are true:

```
controller_verified
AND ENABLED=1
AND ESTOP=0
AND FAULT=NONE
AND BUSY=0
```

Additionally, require XH=1 or ZH=1 before absolute movement of that axis.

14.4 Moving

- Disable conflicting motion controls.
- Keep STOP and ESTOP available.
- Animate from current to target using STATUS values if desired.
- Avoid frequent STATUS polling during motion. Long replies can occupy the MCU UART and introduce STEP timing jitter.
- Prefer one STATUS immediately after acceptance if needed, then wait for **EVENT IDLE**, then request STATUS again.
- Always continue reading asynchronously.

14.5 Error or E-stop

- Display the complete ERR/EVENT line.
- Disable motion controls.
- For software E-stop recovery, require operator confirmation, send CLEAR, send ENABLE, request STATUS, and normally require re-zeroing.

15. Recommended polling and timeout policy

- PING handshake timeout: approximately 2.5 s.
- Idle STATUS polling: 250 to 500 ms is reasonable.
- Busy STATUS polling: avoid continuous polling; rely on EVENT IDLE.
- Never stop the receive loop while waiting for an acknowledgement.
- A movement acknowledgement confirms target acceptance, not physical arrival.
- Physical arrival is represented by EVENT IDLE and a subsequent BUSY=0 STATUS.
- Because there is no checksum, reject malformed or non-ASCII lines locally.
- Preserve a timestamped TX/RX log for commissioning and fault analysis.

Suggested log notation:

```
[12:34:56.100] TX > MOVE_MM X 1 Z 1
[12:34:56.105] RX < OK MOVE_MM
[12:34:56.430] RX < EVENT IDLE X=10 Z=10
[12:34:56.440] TX > STATUS
[12:34:56.465] RX < STATUS ENABLED=1 ... XMM=1.0000 ... ZMM=1.0000 ...
```

16. Complete commissioning example

```

TX > PING
RX < OK PONG LASER_XZ 2.4

TX > STATUS
RX < STATUS ENABLED=0 ESTOP=0 FAULT=NONE BUSY=0 ...

TX > CLEAR
RX < OK CLEAR

TX > ENABLE
RX < OK ENABLE

TX > ZERO ALL
RX < OK ZERO

TX > SPEED_MM ALL 5
RX < OK SPEED_MM

TX > ACCEL_MM ALL 20
RX < OK ACCEL_MM

TX > MOVE_MM X 1 Z 1
RX < OK MOVE_MM
RX < EVENT IDLE X=10 Z=10

TX > STATUS
RX < STATUS ENABLED=1 ESTOP=0 FAULT=NONE BUSY=0 X=10 XT=10 XH=1 XMM=1.0000 XTMM=1.0000 XSPMM=10.0000 XVMM=5.000
XAMM=20.000 Z=10 ZT=10 ZH=1 ZMM=1.0000 ZTMM=1.0000 ZSPMM=10.0000 ZVMM=5.000 ZAMM=20.000

TX > MOVE_MM X -1 Z -1
RX < OK MOVE_MM
RX < EVENT IDLE X=0 Z=0

TX > DISABLE
RX < OK DISABLE

```

17. Minimal host pseudocode

```

open_uart(baud=115200, data_bits=8, parity=none, stop_bits=1)
start_background_line_reader()

send_line("PING")
wait_until(line == "OK PONG LASER_XZ 2.4", timeout=2.5 seconds)
send_line("STATUS")

on_received_line(line):
    if line starts with "STATUS ":
        fields = parse_key_value_fields(line)
        update_gui_state(fields)
    else if line starts with "EVENT IDLE ":
        mark_motion_complete()
        send_line("STATUS")
    else if line starts with "EVENT ESTOP ":
        lock_motion_controls()
        show_estop_state()
    else if line starts with "ERR ":
        lock_or_update_controls_as_required(line)
        show_complete_error(line)
        send_line("STATUS")
    else if line starts with "OK ":
        complete_pending_command(line)
    else if line starts with "READY ":
        record_startup_version(line)

```

Do not parse STATUS by fixed character offsets. Split on spaces, then split recognized fields at the first equals sign. Keep numeric conversion locale independent.

18. Safety requirements for integrating GUIs

- Keep laser emission disabled or safely terminated during motion testing.
- Start with 0.1 to 1.0 mm jogs, low speed, and low acceleration.

- Provide visible STOP and ESTOP controls, but label serial ESTOP as software control rather than a safety-rated emergency circuit.
- Do not allow a GUI window, animation, or pulse counter to be treated as proof of actual position.
- Fit physical hard stops because firmware has no travel limits.
- Warn before DISABLE because the Z mechanism can fall.
- Do not unplug a motor while driver power is present.
- After any loss of holding torque or known missed step, require re-zeroing.

Appendix A. Quick command card

```

PING
STATUS
HELP
CLEAR
ENABLE
STOP
DISABLE
ESTOP
ZERO X
ZERO Z
ZERO ALL
MOVE_MM X <relative_mm>
MOVE_MM Z <relative_mm>
MOVE_MM X <relative_mm> Z <relative_mm>
GOTO_MM X <absolute_mm>
GOTO_MM Z <absolute_mm>
GOTO_MM X <absolute_mm> Z <absolute_mm>
SPEED_MM X|Z|ALL <mm_per_second>
ACCEL_MM X|Z|ALL <mm_per_second_squared>

```

Appendix B. Current default values

Item	X	Z
Pulses/mm	10	10
mm/pulse	0.1	0.1
Default maximum speed	20 mm/s	20 mm/s
Default acceleration	40 mm/s ²	40 mm/s ²
Enforced speed range	1 to 100 mm/s	1 to 100 mm/s
Enforced acceleration range	5 to 200 mm/s ²	5 to 200 mm/s ²
Conservative real positioning increment	0.2 mm	0.2 mm
Displayed GUI envelope	0 to 200 mm	0 to 200 mm

Appendix C. Basis for the enforced operating envelope

The limits above were selected from the hardware supplied with this project:

- The PBC Linear NEMA 17 datasheet specifies 200 full steps/revolution, +/-5% step accuracy, 2 A maximum rated current for its listed NEMA 17 variants, and continuous-duty torque curves for 12, 24, 36, and 48 V bipolar drives.
- At the enforced 100 mm/s limit, the 20T GT2 mechanism turns at 150 RPM and requires 500 full steps/s. The 12 V curves retain substantially more torque there than at the multi-thousand-full-step/s region.
- The Texas Instruments DRV8825 datasheet specifies an 8.2 to 45 V motor supply, 1.9 microseconds minimum STEP high and low times, and 650 ns DIR setup time. The firmware uses a 4 microsecond STEP-high pulse and 1 microsecond DIR setup.
- The motor curves assume their stated current and a test setup. Actual carrier cooling, current-limit adjustment, moving mass, gravity, wheel preload, belt tension, and cable drag can only be validated on the assembled mechanism.